

Câu 1: Danh sách đặc là danh sách

- A. Các phần tử được lưu trữ kế nhau trong bộ nhớ.
- B. Các phần tử được lưu trữ không liên tiếp trong bộ nhớ.
- C. Các phần tử được lưu trữ kế từng đoạn trong bộ nhớ.
- D. Tất cả các câu đều sai.

Câu 2: Khai báo danh sách đặc như sau:

```
const int MAX = 50;  
struct DAYSO {  
    int n;  
    int ds[MAX];  
};
```

Chọn phát biểu SAI

- A. n là số phần tử tối đa trong danh sách.
- B. ds là mảng 1 chiều lưu trữ các phần tử trong danh sách.
- C. n là độ dài danh sách.
- D. MAX là số phần tử tối đa trong danh sách.

Câu 3: Trong phép toán chèn 1 phần tử vào danh sách, vị trí chèn pos nào sau đây hợp lệ

- A. $0 \leq \text{pos} \leq n$.
- B. $0 < \text{pos} \leq n$.
- C. $0 \leq \text{pos} < n$.
- D. $0 < \text{pos} < n$.

Câu 4: Trong phép toán loại bỏ 1 phần tử khỏi danh sách, vị trí pos nào sau đây hợp lệ

- A. $0 \leq \text{pos} < n$.
- B. $0 < \text{pos} \leq n$.
- C. $0 \leq \text{pos} \leq n$.
- D. $0 < \text{pos} < n$.

Câu 5: Đối với thuật toán tìm kiếm nhị phân

- A. Danh sách phải có thứ tự
- B. Danh sách không cần có thứ tự.
- C. Danh sách phải có thứ tự tăng dần.
- D. Danh sách phải có thứ tự giảm dần.

Câu 6: Đối với thuật toán tìm kiếm tuần tự

- A. Danh sách không cần có thứ tự.
- B. Danh sách phải có thứ tự.
- C. Danh sách phải có thứ tự tăng dần.
- D. Danh sách phải có thứ tự giảm dần.

Câu 7: Các bước để chèn 1 phần tử a vào danh sách đặc tại vị trí pos là

- A. Kiểm tra danh sách còn chỗ và pos hợp lệ
Dịch các phần tử từ vị trí pos đến cuối danh sách sang phải 1 ô
Chèn a vào vị trí pos
Tăng độ dài danh sách lên 1.
- B. Kiểm tra danh sách còn chỗ và pos hợp lệ
Dịch các phần tử từ vị trí pos đến cuối danh sách sang trái 1 ô
Chèn a vào vị trí pos
Tăng độ dài danh sách lên 1.

C. Kiểm tra danh sách còn chỗ và pos hợp lệ

Chèn a vào vị trí pos

Tăng độ dài danh sách lên 1.

D. Kiểm tra danh sách còn chỗ và pos hợp lệ

Dịch các phần tử sau vị trí pos đến cuối danh sách sang phải 1 ô

Chèn a vào vị trí pos

Tăng độ dài danh sách lên 1

Câu 8: Các bước để loại bỏ 1 phần tử khỏi danh sách đặc tại vị trí pos là

A. Kiểm tra pos hợp lệ

Dịch các phần tử từ vị trí pos+1 đến cuối danh sách sang trái 1 ô

Giảm độ dài danh sách xuống 1.

B. Kiểm tra pos hợp lệ

Dịch các phần tử từ vị trí pos+1 đến cuối danh sách sang phải 1 ô

Giảm độ dài danh sách xuống 1.

C. Kiểm tra pos hợp lệ

Dịch các phần tử từ vị trí pos đến cuối danh sách sang trái 1 ô

Giảm độ dài danh sách xuống 1.

D. Tất cả các câu đều sai.

Câu 9: Điền khuyết code (1), code (2) của thuật toán tìm nhị phân cho dãy sắp giảm dần

```
struct DAYSO {  
    int n;  
    int ds[20];  
};  
int timNhiPhan(DAYSO x, int a) {  
    int bottom = 0, top = x.n - 1; int mid;  
    do {  
        mid = (bottom + top) / 2;  
        if (x.ds[mid] == a) return mid;  
        else if (a < x.ds[mid]) (1)  
        else (2)  
    } while (bottom <= top);  
  
    return -1;  
}
```

A. (1) bottom = mid+1; (2) top = mid-1;

B. (1) top = mid-1; (2) bottom = mid+1;

C. (1) bottom = mid-1; (2) top = mid+1;

D. (1) top = mid+1; (2) bottom = mid-1;

Câu 10: Cho biết ý nghĩa hàm F sau :

```
struct DAYSO {  
    int n;  
    int ds[20];  
};  
int F(DAYSO x, int a) {  
    int vt = -1;  
    for (int i = 0; i < x.n; i++)  
        if (x.ds[i] == a) vt = i;  
  
    return vt;  
}
```

A. Hàm tìm kiếm tuần tự trả về vị trí xuất hiện sau cùng của phần tử a nếu tìm thấy, trả về -1 nếu không tìm thấy.

- B. Hàm tìm kiếm tuần tự trả về vị trí xuất hiện đầu tiên của phần tử a nếu tìm thấy, trả về -1 nếu không tìm thấy.
- C. Hàm tìm kiếm nhị phân trả về vị trí xuất hiện sau cùng của phần tử a nếu tìm thấy, trả về -1 nếu không tìm thấy.
- D. Hàm tìm kiếm nhị phân trả về vị trí xuất hiện đầu tiên của phần tử a nếu tìm thấy, trả về -1 nếu không tìm thấy.

Câu 11: Danh sách liên kết là danh sách mà các phần tử

- A. Có ít nhất 2 thành phần: dữ liệu, phần liên kết
- B. Có tối đa 2 thành phần: dữ liệu, phần liên kết
- C. Có 3 thành phần: dữ liệu, liên kết prev, liên kết next.
- D. Có 4 thành phần: dữ liệu, liên kết link, liên kết prev, liên kết next.

Câu 12: Cho cấu trúc danh sách liên kết đơn, giả sử đã có hàm tạo node.

```
typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
NODEPTR taoNode(TYPEDATA a);
void themDau(NODEPTR& list, TYPEDATA a) {
}
}
```

Code cho hàm themDau thêm phần tử vào đầu danh sách là:

- A. `NODEPTR p = taoNode(a);`
`p->link = list;`
`list = p;`
- B. `NODEPTR p = taoNode(a);`
`list = p;`
`p->link = list;`
- C. `NODEPTR p = list;`
`p->link = list;`
`list = p;`
- D. `NODEPTR p = list->link;`
`list = p;`
`p->link = list;`

Câu 13: Cho cấu trúc danh sách liên kết đơn.

```
typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
NODEPTR timPhanTuCuoi(NODEPTR list) {
}
}
```

Code cho hàm timPhanTuCuoi tìm phần tử cuối danh sách là:

- A. `NODEPTR q = list;`
`while (q != NULL && q->link != NULL) q = q->link;`
`return q;`
- B. `NODEPTR q = list;`
`while (q != NULL || q->link != NULL) q = q->link;`
`return q;`
- C. `NODEPTR q = list;`

```

while (q != NULL) q = q->link;
return q;
D. NODEPTR q = list;
while (q->link != NULL) q = q->link;
return q;

```

Câu 14: Cho cấu trúc danh sách liên kết đơn.

```

typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
NODEPTR timMax(NODEPTR list) {
}

```

Code cho hàm timMax tìm phần tử lớn nhất danh sách là:

- A.

```

NODEPTR p = list, max = list;
while (p != NULL) {
    if (p->data > max->data)
        max = p;
    p = p->link;
}
return p;

```
- B.

```

NODEPTR p = list, max = p->data;
while (p != NULL) {
    if (p->data > max->data)
        max = p;
    p = p->link;
}
return p;

```
- C.

```

NODEPTR p = list, max = list;
while (p != NULL) {
    if (p->data > max->data)
        max->data = p->data;
    p = p->link;
}
return p;

```
- D.

```

NODEPTR p = list, max = list;
while (p != NULL) {
    if (p->data > max->data)
        max = p->data;
    p = p->link;
}
return p;

```

Câu 15: Để xóa 1 phần tử (không phải phần tử đầu) trong danh sách liên kết đơn, ta phải tìm

- A. Phần tử đứng trước nó.
- B. Phần tử đứng sau nó.
- C. Phần tử đầu danh sách.
- D. Phần tử cuối danh sách.

Câu 16: Phần liên kết của phần tử cuối trong danh sách liên kết vòng

- A. Trở đến phần tử đầu.
- B. Trở đến phần tử NULL.
- C. Trở đến phần tử kề trước.
- D. Trở đến phần tử kề sau.

Câu 17: Cấu trúc một node của danh sách liên kết kép gồm:

- A. Phần dữ liệu, phần liên kết phần tử kế trước, phần liên kết phần tử kế sau.
- B. Phần dữ liệu, phần liên kết phần tử kế trước, phần liên kết phần tử đầu.
- C. Phần dữ liệu, phần liên kết phần tử kế sau, phần liên kết phần tử cuối.
- D. Phần dữ liệu, phần liên kết chính nó, phần liên kết phần tử sau.

Câu 18: Cho cấu trúc danh sách liên kết kép như sau:

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* next;
    NODE* prev;
};
typedef NODE* NODEPTR;
struct DLIST {
    NODEPTR first;
    NODEPTR last;
};
void duyetToi(DLIST dlist) {
}
```

Code cho hàm duyetToi là:

- A.

```
NODEPTR p = dlist.first;
while (p != NULL) {
    cout << " " << p->data;
    p = p->next;
}
```
- B.

```
NODEPTR p = dlist.first;
while (p != NULL) {
    cout << " " << p->data;
    p = p->prev;
}
```
- C.

```
NODEPTR p = dlist.last;
while (p != NULL) {
    cout << " " << p->data;
    p = p->next;
}
```
- D.

```
NODEPTR p = dlist.last;
while (p != NULL) {
    cout << " " << p->data;
    p = p->prev;
}
```

Câu 19: Cho cấu trúc danh sách liên kết kép như sau:

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* next;
    NODE* prev;
};
typedef NODE* NODEPTR;
struct DLIST {
    NODEPTR first;
    NODEPTR last;
};
void duyetLui(DLIST dlist) {
}
```

Code cho hàm duyệtLui là:

- A. `NODEPTR p = dlist.last;`
 `while (p != NULL) {`
 `cout << " " << p->data;`
 `p = p->prev;`
 `}`
- B. `NODEPTR p = dlist.last;`
 `while (p != NULL) {`
 `cout << " " << p->data;`
 `p = p->next;`
 `}`
- C. `NODEPTR p = dlist.first;`
 `while (p != NULL) {`
 `cout << " " << p->data;`
 `p = p->next;`
 `}`
- D. `NODEPTR p = dlist.first;`
 `while (p != NULL) {`
 `cout << " " << p->data;`
 `p = p->prev;`
 `}`

Câu 20: Cho cấu trúc danh sách liên kết kép như sau:

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* next;
    NODE* prev;
};
typedef NODE* NODEPTR;
struct DLIST {
    NODEPTR first;
    NODEPTR last;
};
void khoiTao(DLIST& dlist) {
}
```

Code cho hàm khoiTao là:

- A. `dlist.first = NULL; dlist.last = NULL;`
- B. `dlist->first = NULL; dlist->last = NULL;`
- C. `dlist.first->next = NULL; dlist.last->prev = NULL;`
- D. `dlist.next = NULL; dlist.prev = NULL;`

Câu 21: Phép toán POP dùng trong stack để:

- A. Lấy phần tử tại đỉnh ra khỏi stack.
- B. Xóa phần tử bất kì khỏi stack.
- C. Xóa bỏ một dãy các phần tử ra khỏi stack
- D. Xóa phần tử cuối cùng khỏi stack.

Câu 22: Phép toán PUSH dùng trong stack để:

- A. Thêm một phần tử vào đỉnh stack.
- B. Thêm một phần tử vào vị trí bất kì của stack.
- C. Thêm một dãy các phần tử vào đỉnh của stack
- D. Thêm một phần tử vào cuối stack.

Câu 23: Stack là danh sách dạng

- A. LIFO
- B. FIFO
- C. FILO
- D. LILO

Câu 24: Để lấy giá trị tại đỉnh của stack nhưng không loại bỏ phần tử ta dùng phép toán:

- A. TOP
- B. POP
- C. PUSH
- D. EMPTY

Câu 25: Cho danh sách {1, 2, 4, 3, 11, 7, 8}, Top ở phần tử 8, sau khi thực hiện POP 3 lần, stack còn lại là:

- A. 1, 2, 4, 3
- B. 3, 11, 7, 8
- C. 1, 2, 4, 11, 7, 8
- D. 3

Câu 26: Phép toán DeQueue dùng trong hàng đợi để:

- A. Loại bỏ phần tử ở đầu hàng.
- B. Loại bỏ phần tử ở cuối hàng
- C. Xóa bỏ một dãy các phần tử ra khỏi hàng
- D. Xóa phần tử ở vị trí bất kỳ trong hàng.

Câu 27: Phép toán EnQueue dùng trong hàng đợi để:

- A. Thêm một phần tử vào cuối hàng.
- B. Thêm một phần tử vào vị trí bất kỳ của hàng.
- C. Thêm một dãy các phần tử vào hàng.
- D. Thêm một phần tử vào vị trí bất kỳ trong hàng.

Câu 28: Queue là danh sách dạng

- A. FIFO
- B. LIFO
- C. FILO
- D. LILO

Câu 29: Để lấy giá trị đầu hàng đợi nhưng không loại bỏ phần tử ta dùng phép toán:

- A. FRONT
- B. ENQUEUE
- C. DEQUEUE
- D. EMPTY

Câu 30: Cho danh sách {1, 2, 5, 8}, Front ở phần tử 8, Rear ở phần tử 1, sau khi thực hiện ENQUEUE(9) 2 lần, hàng có các phần tử là:

- A. 9, 9, 1, 2, 5, 8
- B. 1, 2, 5, 8, 9, 9
- C. 1, 2, 9, 9, 5, 8
- D. 9, 1, 2, 5, 8, 9

Câu 31: Cho cấu trúc danh sách liên kết đơn, và hàm hủy danh sách

```
typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
void huy(NODEPTR& list) {
    NODEPTR p;
    while (list != NULL) {
        (1)
        (2)
        (3)
    }
    list = NULL;
}
```

Viết các câu lệnh (1), (2), (3)

- A. (1) `p = list;` (2) `list = list->link;` (3) `delete p;`
- B. (1) `list = p;` (2) `list = list->link;` (3) `delete p;`
- C. (1) `list = p->link;` (2) `p = list;` (3) `delete p;`
- D. (1) `p = list;` (2) `delete p;` (3) `list = list->link;`

Câu 32: Cho cấu trúc danh sách liên kết đơn, và hàm tìm vị trí phần tử a

```
typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
//Tim vi tri dau tien cua phan tu a
int timViTri(NODEPTR list, TYPEINFO a) {
    int vt = -1;
    NODEPTR p = list;
    while (p != NULL) {
        (1)
        (2)
        (3)
    }
    return vt;
}
```

- A. (1) `vt++;` (2) `if (p->data == a) break;` (3) `p = p->link;`
- B. (1) `vt++;` (2) `if (p->data == a) return p;` (3) `p = p->link;`
- C. (1) `vt++;` (2) `if (p->data == a) break;` (3) `p = p->link;`
- D. (1) `vt++;` (2) `if (p->data == a) return p;` (3) `p = p->link;`

Câu 33: Cho cấu trúc danh sách liên kết đơn, và hàm F

```
typedef int TYPEDATA;
struct NODE {
    TYPEDATA data;
    NODE* link;
};
typedef NODE* NODEPTR;
void F(NODEPTR& list) {
    if (list != NULL) {
        NODEPTR p = list;
        list = list->link;
        delete p;
    }
}
```

- A. Hàm F dùng để xóa phần tử đầu danh sách
- B. Hàm F dùng để xóa phần tử cuối danh sách
- C. Hàm F dùng để xóa phần tử list
- D. Hàm F dùng để xóa phần tử p

Câu 34: Cho cấu trúc Queue, và hàm K

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* link;
};
typedef NODE* NODEPTR;
struct QUEUE {
    NODEPTR front;
    NODEPTR rear;
};
int K(QUEUE q, TYPEINFO& a) {
    if (!IsEmpty(q)) {
        a = q.front->data;
        return 1;
    }
    return 0;
}
```

- A. Hàm K là phép toán FRONT, lấy phần đầu hàng nhưng không loại bỏ.
- B. Hàm K là phép toán FRONT, lấy phần đầu hàng và loại bỏ khỏi hàng.
- C. Hàm K là phép toán DEQUEUE, lấy phần đầu hàng nhưng không loại bỏ.
- D. Hàm K là phép toán DEQUEUE, lấy phần đầu hàng và loại bỏ khỏi hàng.

Câu 35: Cho cấu trúc Stack, giả sử đã có hàm taoNode và hàm H

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* link;
};
typedef NODE* NODEPTR;
struct STACK {
    NODEPTR top;
};
NODEPTR taoNode(TYPEINFO a);
void H(STACK& s, TYPEINFO a) {
    NODEPTR p = taoNode(a);
    p->link = s.top;
    s.top = p;
}
```

- A. Hàm H là phép toán PUSH
- B. Hàm H là phép toán POP
- C. Hàm H là phép toán TOP
- D. Hàm H là phép toán EMPTY

Câu 36: Cho cấu trúc DAYSO thể hiện danh sách đặc, và hàm chèn phần tử tại vị trí pos

```
const int MAX = 50;
struct DAYSO {
    int n;
    int ds[MAX];
};
void chen(DAYSO& x, int pos, int a) {
    //(1) Kiểm tra danh sách chưa đầy và vị trí pos hợp lệ
```

```

    (2)
    (3)
    (4)
}
Viết các câu lệnh ở Bước 2, Bước 3, Bước 4
A. (2) for (int i = x.n; i > pos; i--) x.ds[i] = x.ds[i - 1];
(3) x.ds[pos] = a;
(4) x.n++;
B. (2) for (int i = x.n-1; i > pos; i--) x.ds[i] = x.ds[i - 1];
(3) x.ds[pos] = a;
(4) x.n++;
C. (2) for (int i = x.n; i >= pos; i--) x.ds[i] = x.ds[i - 1];
(3) x.ds[pos] = a;
(4) x.n++;
D. (2) for (int i = x.n-1; i > pos; i--) x.ds[i] = x.ds[i - 1];
(3) x.ds[pos] = a;
(4) x.n++;

```

Câu 37: Ưu điểm của danh sách đặc

- A. Dễ cài đặt.
- B. Tốc độ truy xuất nhanh (thông qua chỉ số)
- C. Thích hợp các phép toán tìm kiếm.
- D. Tất cả các câu đều đúng.

Câu 38: Ưu điểm danh sách liên kết

- A. Sử dụng bộ nhớ tốt hơn (không bị phân mảnh)
- B. Thích hợp các phép toán thêm vào và loại bỏ
- C. Câu A, B đúng.
- D. Câu A, B sai.

Câu 39: Cấu trúc dữ liệu cho phép thêm và loại bỏ phần tử ở đầu là cấu trúc

- A. Stack
- B. Queue
- C. Tree
- D. List

Câu 40: Cấu trúc dữ liệu cho phép thêm ở cuối danh sách, lấy ra ở đầu danh sách là cấu trúc

- A. Queue
- B. Stack
- C. Tree
- D. List

Câu 41: Ý tưởng của thuật toán sắp xếp một mảng tăng dần bằng cách đi tìm phần tử có giá trị nhỏ nhất trong đoạn chưa được sắp xếp và đổi chỗ phần tử nhỏ nhất đó với phần tử ở đầu đoạn chưa được sắp xếp là thuật toán.

- A. Selection Sort
- B. Insertion Sort
- C. Bubble Sort
- D. Heap Sort

Câu 42: Cho mảng $A = \{3, 9, 2, 18, 23, 32, 4, 9, 10\}$, mảng kết quả sau khi chạy từng bước sắp xếp mảng A tăng dần bằng phương pháp chọn trực tiếp sau 3 lần lặp là:

- A. 2, 3, 4, 18, 23, 32, 9, 9, 10
- B. 2, 3, 4, 9, 23, 32, 18, 9, 10
- C. 2, 9, 3, 18, 23, 32, 4, 9, 10
- D. 2, 3, 9, 18, 23, 32, 4, 9, 10

Câu 43: Ý tưởng sắp xếp một mảng tăng dần bằng cách duyệt từng phần tử và chèn từng phần tử đó vào đúng vị trí trong mảng con (dãy số từ đầu đến phần tử phía trước nó) đã sắp xếp sao cho dãy số trong mảng sắp đã xếp đó vẫn đảm bảo tính chất của một dãy số tăng dần.

- A. Insertion Sort
- B. Quick Sort
- C. Bubble Sort
- D. Heap Sort

Câu 44: Cho mảng $A = \{3, 9, 2, 18, 23, 32, 4, 9, 10\}$, mảng kết quả sau khi chạy từng bước sắp xếp mảng A tăng dần bằng phương pháp chèn trực tiếp sau 3 lần lặp là:

- A. 2, 3, 9, 18, 23, 32, 4, 9, 10
- B. 3, 9, 2, 18, 23, 32, 4, 9, 10
- C. 2, 3, 4, 9, 18, 23, 32, 9, 10
- D. 2, 3, 4, 9, 9, 18, 23, 32, 10

Câu 45: Ý tưởng sắp xếp một mảng tăng dần bằng cách lặp lại công việc đổi chỗ 2 số liên tiếp nhau nếu chúng đứng sai thứ tự (số sau bé hơn số trước) cho đến khi dãy số được sắp xếp.

- A. Bubble Sort
- B. Quick Sort
- C. Selection Sort
- D. Heap Sort

Câu 46: Cho mảng $A = \{3, 9, 2, 18, 23, 32, 4, 8, 10, 1\}$, mảng kết quả sau khi chạy từng bước sắp xếp mảng A tăng dần bằng phương pháp nổi bọt sau 3 lần lặp là:

- A. 2, 3, 9, 4, 8, 10, 1, 18, 23, 32
- B. 2, 3, 9, 18, 4, 8, 10, 1, 23, 32
- C. 2, 3, 4, 8, 9, 1, 10, 18, 23, 32
- D. 3, 2, 9, 18, 23, 4, 8, 10, 1, 32

Câu 47: Trong thuật toán sắp xếp vun đống (heap sort), để sắp xếp một mảng tăng dần, ta cần xây dựng một heap sao cho:

- A. Phần tử thứ i lớn hơn hay bằng phần tử liên đới $2i + 1$ và $2i + 2$ ($i = 0..n/2$)
- B. Phần tử thứ i nhỏ hơn hay bằng phần tử liên đới $2i + 1$ và $2i + 2$ ($i = 0..n/2$)
- C. Phần tử thứ i lớn hơn phần tử liên đới $2i + 1$ và $2i + 2$ ($i = 0..n/2$)
- D. Phần tử thứ i nhỏ hơn phần tử liên đới $2i + 1$ và $2i + 2$ ($i = 0..n/2$)

Câu 48: Cho mảng $A = \{3, 9, 2, 18, 23, 32, 4, 8, 10, 1\}$, kết quả sau khi dùng thuật toán heap sort để hiệu chỉnh mảng A thành một heap : (trường hợp sắp tăng dần)

- A. 32, 23, 4, 18, 9, 2, 3, 8, 10, 1
- B. 1, 23, 4, 18, 9, 2, 3, 8, 10, 32
- C. 23, 18, 4, 10, 9, 2, 3, 8, 1, 32
- D. 1, 18, 4, 10, 9, 2, 3, 8, 23, 32

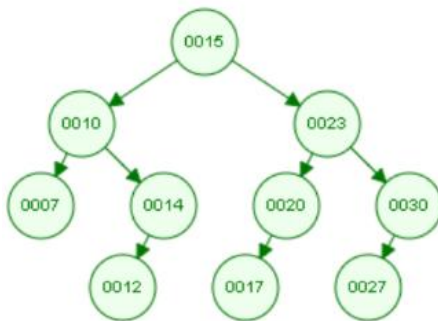
Câu 49: Ý tưởng sắp xếp một mảng tăng dần bằng cách chọn một phần tử trong mảng làm phần tử pivot (phần tử chốt), sau đó thực hiện chia mảng thành các mảng con (A1 các phần tử nhỏ hơn pivot, A2 các phần tử bằng pivot, A3 các phần tử lớn hơn pivot) dựa vào phần tử pivot đã chọn.

- A. Quick Sort
- B. Bubble Sort
- C. Selection Sort
- D. Heap Sort

Câu 50: Cho mảng $A = \{3, 9, 2, 18, 23, 32, 4, 8, 10, 6\}$, kết quả sau khi dùng thuật toán quick sort để phân hoạch mảng A, chọn phần tử cuối ($A[9] = 6$) làm pivot (cho trường hợp mảng sắp tăng dần)

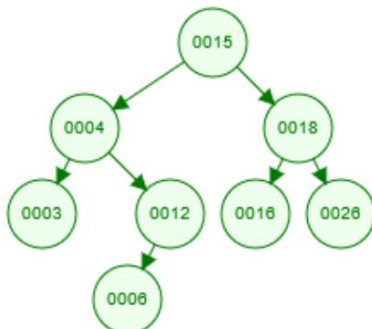
- A. 3, 4, 2, 6, 23, 32, 9, 8, 10, 18
- B. 3, 4, 2, 18, 23, 32, 9, 8, 10, 6
- C. 6, 4, 2, 3, 23, 32, 9, 8, 10, 18
- D. 6, 4, 2, 18, 23, 32, 9, 8, 10, 3

Câu 51: Chọn thứ tự nhập vào dãy số để có cây tìm kiếm nhị phân như hình:



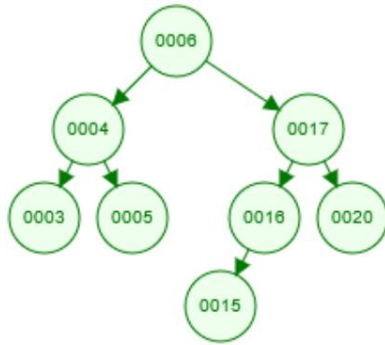
- A. 15, 23, 10, 20, 14, 7, 30, 27, 17, 12
- B. 15, 14, 23, 10, 20, 7, 30, 27, 17, 12
- C. 15, 20, 14, 10, 23, 7, 30, 27, 17, 12
- D. 15, 10, 30, 20, 14, 23, 7, 27, 17, 12

Câu 52: Chọn kết quả duyệt trước (Node Left Right) của cây tìm kiếm nhị phân sau:



- A. 15, 4, 3, 12, 6, 18, 16, 26
- B. 15, 4, 3, 6, 12, 18, 16, 26
- C. 15, 4, 3, 12, 6, 16, 18, 26
- D. 3, 4, 6, 12, 15, 16, 18, 26

Câu 53: Chọn kết quả duyệt sau (Left Right Node) của cây tìm kiếm nhị phân sau:



- A. 3, 5, 4, 15, 16, 20, 17, 6
- B. 3, 5, 4, 6, 15, 16, 20, 17
- C. 6, 4, 3, 5, 17, 16, 15, 20
- D. 3, 5, 4, 15, 16, 17, 20, 6

Câu 54: Cho cấu trúc cây nhị phân và hàm f như sau:

```

typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* left;
    NODE* right;
};
typedef NODE* NODEPTR;
NODEPTR f(NODEPTR r) {
    NODEPTR p = r;
    if (p != NULL)
        while (p->left != NULL) p = p->left;
    return p;
}
  
```

Hàm f dùng để làm gì?

- A. Tìm nút trái tận cùng của cây r.
- B. Tìm nút phải tận cùng của cây r.
- C. Tìm nút 1 con bên trái của cây r.
- D. Tìm nút lá bên trái của cây r.

Câu 55: Cho cấu trúc cây nhị phân và hàm f như sau:

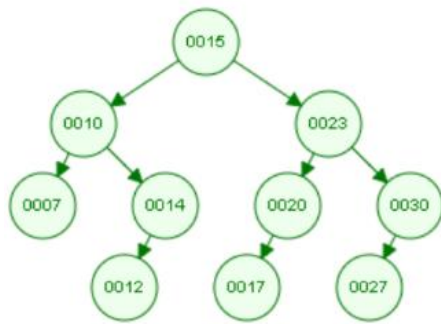
```

typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* left;
    NODE* right;
};
typedef NODE* NODEPTR;
NODEPTR f(NODEPTR r) {
    NODEPTR p = r;
    if (p != NULL)
        while (p->right != NULL) p = p->right;
    return p;
}
  
```

Hàm f dùng để làm gì?

- A. Tìm nút phải tận cùng của cây r.
- B. Tìm nút trái tận cùng của cây r.
- C. Tìm nút 1 con bên phải của cây r.
- D. Tìm nút lá bên phải của cây r.

Câu 56: Cho cây tìm kiếm nhị phân như hình



Để xóa nút 15, hai nút “thế mạng” có thể dùng là:

- A. 14, 17
- B. 12, 17
- C. 10, 23
- D. 7, 27

Câu 57: Phương pháp duyệt cây thích hợp để hủy cây là:

- A. Duyệt sau
- B. Duyệt trước
- C. Duyệt giữa
- D. Tất cả các phương pháp duyệt cây.

Câu 58: Cho cấu trúc cây tìm kiếm nhị phân và hàm tìm

```

typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* left;
    NODE* right;
};
typedef NODE* NODEPTR;
NODEPTR tim(NODEPTR r, TYPEINFO a) {
    if (r != NULL) {
        (1)
        (2)
        (3)
    }
    return NULL;
}
  
```

Mã (1), (2), (3) là:

- A. (1) `if (r->data == a) return r;`
 (2) `else if (a < r->data) return tim(r->left, a);`
 (3) `else return tim(r->right, a);`
- B. (1) `if (r->data == a) return r;`
 (2) `else if (a < r->data) return tim(r->right, a);`
 (3) `else return tim(r->left, a);`
- C. (1) `if (r->data = a) return r;`
 (2) `else if (a < r->data) return tim(r->left, a);`
 (3) `else return tim(r->right, a);`
- D. (1) `if (r->data == a) return r;`
 (2) `else if (a < r->data) return r->left;`
 (3) `else return r->right;`

Câu 59: Cho cấu trúc cây nhị phân và hàm k

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* left;
    NODE* right;
};
typedef NODE* NODEPTR;
int k(NODEPTR r) {
    if (r != NULL) {
        int a = k(r->left);
        int b = k(r->right);
        if (a > b) b = a;
        return b + 1;
    }
    return 0;
}
```

Hàm k dùng để:

- A. Tính chiều cao của cây r.
- B. Tính mức của nút r.
- C. Đếm số nút đủ 2 con.
- D. Đếm số nút lá.

Câu 60: Cho cấu trúc cây nhị phân và hàm t

```
typedef int TYPEINFO;
struct NODE {
    TYPEINFO data;
    NODE* left;
    NODE* right;
};
typedef NODE* NODEPTR;
int t(NODEPTR r) {
    if (r != NULL) {
        return 1 + t(r->left) + t(r->right);
    }
    return 0;
}
```

Hàm t dùng để

- A. Đếm số nút trong cây.
- B. Tính chiều cao cây.
- C. Đếm số nút lá.
- D. Đếm số nút trung gian.